

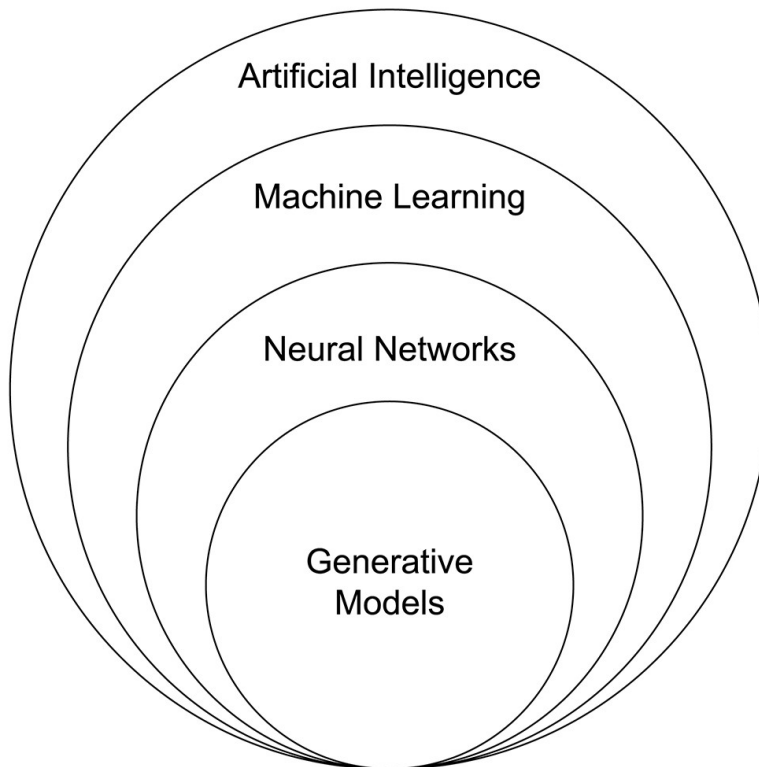
# Generative AI for Mechanical Engineering

## Part II: NLP Libraries, Hugging Face, APIs, and Simple Mechanical Engineering AI Workflows

A beginner-friendly tutorial for students with introductory Python experience, with some overlapping with Part I of last Module for review and clarity

Prepared for classroom use in current Google Colab-style environments

This document introduces practical generative artificial intelligence workflows for Mechanical Engineering students who are new to natural language processing, multimodal artificial intelligence interfaces, and beginner application programming interface use. It emphasizes practical input-output behavior, small proof-of-concept workflows, careful interpretation, and responsible engineering use.



*Figure 1. Generative artificial intelligence sits inside the broader fields of artificial intelligence, machine learning, and neural networks. This hierarchy helps students see that API-based tools and small local workflows are part of a larger technical family. Source: The Original Benny C (2023), Wikimedia Commons, CC BY-SA 4.0. Citation: The Original Benny C, 2023.*

## Table of Contents

- Detailed Teaching Outline
- 1. Introduction to Generative AI in Mechanical Engineering
- 2. Python and AI Workflow Mounting bases
- 3. Part I: NLP Libraries and Hugging Face
- 4. Introduction to spaCy and NLTK
- 5. Theory and Background for Beginner NLP Workflows
- 6. Introduction to Hugging Face
- 7. Hugging Face Task-Interface Understanding
- 8. Introductory Hugging Face Multimodal Task Patterns
- 9. Part II: APIs and Simple Mechanical Engineering AI Workflows
- 10. Google and OpenAI APIs for Beginner Mechanical Engineering Workflows
- 11. Simple Mechanical Engineering Applications of AI and Machine Learning
- 12. Beginner Workflow Integration
- 13. Theory and Conceptual Background for Multimodal and API Workflows
- 14. Responsible Use, Project Checklist, and Practice Activities
- 15. References

## Detailed Teaching Outline

The teaching sequence begins with the material objects that students can see and run: strings, images, tables, prompts, responses, and short Python programs. It then moves from rule-based natural language processing to pretrained task interfaces, then to hosted application programming interfaces, and finally to small integrated Mechanical Engineering proof-of-concept workflows.

**Table 1. Improved beginner-first teaching sequence. Source: Synthesized for this tutorial from official spaCy, NLTK, Hugging Face, TensorFlow, Google, and OpenAI documentation cited in the references.**

| Stage                    | Main idea                                                                              | Student activity                                                   | Mechanical Engineering link                                                       |
|--------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Orientation              | Generative AI can produce, summarize, transform, or describe data.                     | Compare predictive and generative examples.                        | Connect inspection notes, site photos, audio comments, and short reports.         |
| Python mounting bases    | Inputs and outputs are represented as strings, arrays, tables, and objects.            | Run compact notebook cells and inspect printed results.            | Prepare notes, images, and tabular records before using AI.                       |
| spaCy and NLTK           | Simple rules and tokens can extract useful structure from text.                        | Clean notes, tokenize text, stem words, and match entity patterns. | Extract asset type, defect type, and severity terms from inspection comments.     |
| Hugging Face             | A task interface hides much of the complexity of pretrained models.                    | Run or inspect text, image, audio, and video task examples.        | Caption inspection images, transcribe field audio, or classify short media clips. |
| API workflows            | A hosted model receives a request and returns a response object.                       | Send text prompts with graceful missing-key handling.              | Summarize inspection data or draft plain-language maintenance statements.         |
| Local TensorFlow support | Small local models can classify text, images, or tables.                               | Train tiny models and inspect outputs.                             | Create proof-of-concept condition screening or priority grouping.                 |
| Integration              | Practical workflows combine preprocessing, local prediction, and hosted summarization. | Build an end-to-end mini project.                                  | Turn raw engineering inputs into a compact summary for review.                    |

**Table 2. Required learning outcomes and where they are addressed. Source: Synthesized for this tutorial from the stated course goals and official tool documentation.**

| Learning outcome                                                        | Where it is addressed         | Evidence students should produce                                                             |
|-------------------------------------------------------------------------|-------------------------------|----------------------------------------------------------------------------------------------|
| Use beginner-friendly NLP libraries and Hugging Face workflows.         | Sections 4 through 8.         | Working code cells for tokenization, rule-based extraction, and several task interfaces.     |
| Build lightweight Mechanical Engineering proof-of-concept applications. | Sections 11 and 12.           | Small TensorFlow classifiers and an integrated mini-project workflow.                        |
| Interpret inputs, outputs, and access options for practical use.        | Sections 2, 6, 7, 10, and 13. | Written explanations of prompts, responses, local inference, hosted inference, and API keys. |

# 1. Introduction to Generative AI in Mechanical Engineering

Generative artificial intelligence, or generative AI, refers to artificial intelligence systems that can produce new outputs from learned patterns. These outputs may be text, images, audio, video, code, tables, or combinations of several data types. A generative system may write a summary, describe an image, transcribe speech, generate a maintenance recommendation draft, or transform a rough prompt into a structured response. This tutorial treats generative AI as a practical set of workflows, not as a single product.

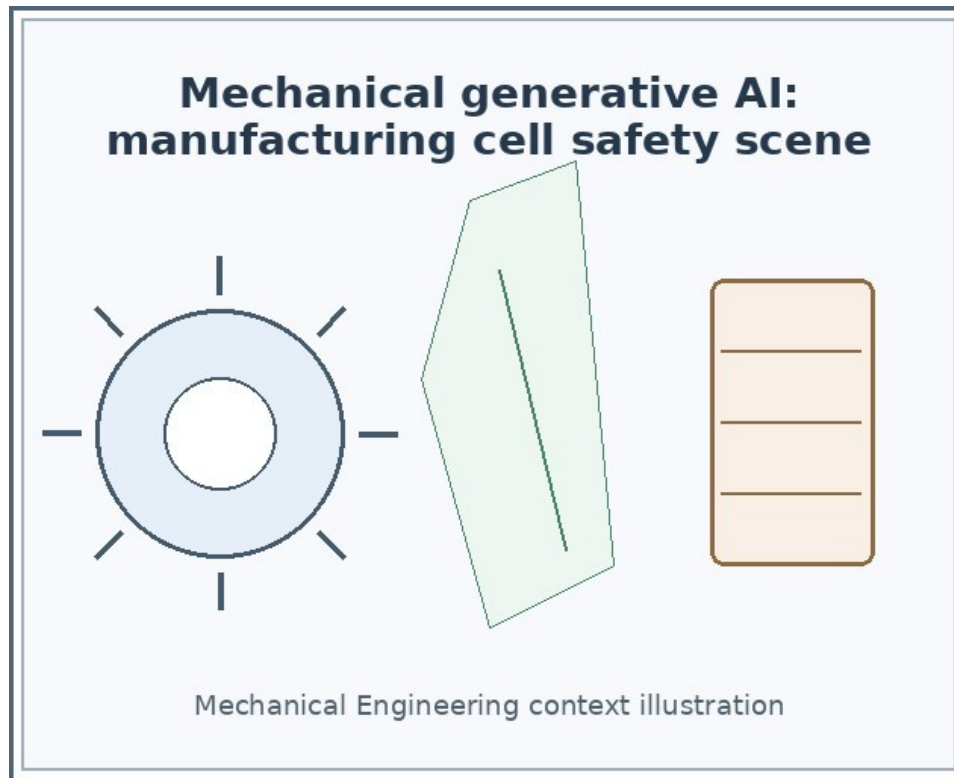
For Mechanical Engineering students, the key question is not whether a model sounds impressive. The key question is: what input did the model receive, what output did it produce, how should that output be checked, and what engineering decision should remain under human professional review?

**Table 3. Predictive AI and generative AI compared in beginner language. Source: Synthesized from OpenAI text-generation documentation, Google Gemini API documentation, Hugging Face task pages, and TensorFlow tutorials.**

| Workflow type   | Beginner description                                                                       | Typical input                                        | Typical output                                                            | Mechanical Engineering example                                                    |
|-----------------|--------------------------------------------------------------------------------------------|------------------------------------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Predictive AI   | Predicts a label, score, class, or value.                                                  | Features, images, notes, or sensor measurements.     | A class such as urgent or normal, or a number such as predicted strength. | Classify inspection notes into low, medium, and high priority.                    |
| Generative AI   | Produces new text, images, audio, video, or structured content based on a prompt or input. | Prompt, image, audio, video, table, or mixed inputs. | Summary, caption, response, transcript, draft report, or generated media. | Draft a plain-language maintenance summary from inspection notes.                 |
| Hybrid workflow | Uses prediction first, then generation to explain or summarize.                            | Raw data plus a prompt.                              | A prediction and a human-readable explanation.                            | A small classifier flags severe notes, then an API drafts a review-ready summary. |

**Table 4. Data types that generative AI systems can process. Source: Synthesized from Hugging Face task documentation, OpenAI API documentation, and Google Gemini API documentation.**

| Data type  | Input example                                                     | Possible generated or transformed output                            | Engineering use case                                                              |
|------------|-------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Text       | Inspection notes, maintenance logs, specifications.               | Summary, rewritten explanation, classification, checklist.          | Summarize a week of mechanical component inspection comments.                     |
| Image      | Site photo, machined surface image, crack image, drawing excerpt. | Caption, condition description, image enhancement, screening score. | Caption a site photo for quick record keeping.                                    |
| Audio      | Field voice memo or meeting note.                                 | Transcript, spoken response, audio summary.                         | Transcribe a spoken observation after a site walk.                                |
| Video      | Short clip of maintenance activity or asset condition.            | Classification, description, key-frame summary.                     | Screen short clips for work activity category or visible motion.                  |
| Multimodal | Image plus text prompt, table plus note, video plus question.     | Answer, explanation, matched description, report draft.             | Connect a inspection image with an inspection note and produce a compact summary. |



*Figure 2. A complete mechanical component inspection photograph showing visible cracking. Mechanical Engineering AI workflows often begin with raw, imperfect field data such as inspection photographs and notes. Source: National Vehicle systems Safety Board (2018), Wikimedia Commons, public domain. Citation: NTSB, 2018.*

Generative AI matters in Mechanical Engineering because engineering organizations already store large amounts of unstructured information: free-text notes, photographs, videos, audio observations, sketches, drawings, and tabular asset records. These data are often useful but difficult to search consistently. Beginner AI workflows can help students learn how to clean, tag, summarize, and interpret such data while keeping final engineering judgment in human hands.

*Table 5. Realistic beginner use cases for Mechanical Engineering students. Source: Synthesized from common Mechanical Engineering data workflows and official task descriptions from Hugging Face, Google, and OpenAI documentation.*

| Use case                      | Beginner workflow                                                                    | Output to inspect                                   | Important limitation                                         |
|-------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------------|
| Inspection-note summarization | Clean notes with spaCy or NLTK, then send a prompt to an API.                        | Short bullet summary for review.                    | The summary can omit or misinterpret details unless checked. |
| Defect keyword extraction     | Use rule-based patterns for words such as crack, spall, corrosion, leak, deflection. | A small table of defect terms and severity words.   | Rules miss unexpected wording and spelling variations.       |
| Image captioning              | Use an image-to-text interface on a site or asset image.                             | A generated caption.                                | Captioning is not a certified inspection decision.           |
| Plain-language reporting      | Turn technical notes into nontechnical maintenance language.                         | Draft paragraph for stakeholder communication.      | Professional review is required before sharing.              |
| Compact dashboard narration   | Combine model outputs with prompt-based text generation.                             | A short narrative explaining counts and priorities. | Numbers must come from verified data, not invented text.     |
| Simple multimodal report      | Combine image caption, note extraction, and summary prompt.                          | One compact report-style output.                    | Multimodal outputs can be persuasive but wrong.              |

**Section summary:** Generative AI can support text, image, audio, video, and mixed-data workflows. In this tutorial, every workflow is treated as a beginner proof of concept. Students should always inspect inputs, outputs, and assumptions before drawing engineering conclusions.

## 2. Python and AI Workflow Mounting bases

This section introduces only the Python ideas needed for the tutorial. The examples assume beginner Python knowledge and use notebook-style cells. A notebook cell can contain installation commands, imports, small data objects, code, and printed output. The most important skill is to read a small code block and identify the input, the operation, and the output.

**Table 6. Python objects used throughout the tutorial. Source: Synthesized from standard Python practice and the code patterns in official TensorFlow, Hugging Face, Google, and OpenAI documentation.**

| Object     | Plain-language meaning            | Small example                                  | How it appears in AI workflows                   |
|------------|-----------------------------------|------------------------------------------------|--------------------------------------------------|
| Variable   | A named value.                    | <code>note = "crack near joint"</code>         | Stores a prompt, note, score, or result.         |
| String     | Text inside quotes.               | <code>"spall on mount"</code>                  | Represents inspection notes and prompts.         |
| List       | An ordered collection.            | <code>["crack", "leak"]</code>                 | Stores many notes or labels.                     |
| Dictionary | Pairs of names and values.        | <code>{"asset": "mechanical component"}</code> | Stores structured outputs and request payloads.  |
| Array      | A grid or sequence of numbers.    | <code>np.array([1, 2, 3])</code>               | Stores image pixels, features, and model inputs. |
| Table      | Rows and columns.                 | <code>pandas DataFrame</code>                  | Stores extracted entities and predictions.       |
| Function   | Reusable instructions.            | <code>clean_text(note)</code>                  | Cleans text or computes a prediction.            |
| Class      | A template for objects.           | <code>SmallReport()</code>                     | Useful for bundling data and methods.            |
| Method     | A function attached to an object. | <code>client.responses.create(...)</code>      | Sends API requests or transforms data.           |
| Argument   | A value passed into a function.   | <code>max_new_tokens=40</code>                 | Controls model behavior.                         |

### Code Example 1. Notebook setup for beginner workflows

Purpose: install and import common libraries. Input: no engineering data yet. Output: a notebook environment ready to run small examples.

```
# These commands are suitable for a Colab-style notebook.
# Run installation commands only when the package is not already available.
!pip install -q spacy nltk pandas matplotlib tensorflow transformers

import os
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

print("Ready for beginner AI workflows.")
```

### Code Example 2. Variables, lists, dictionaries, functions, and a class

Purpose: show the Python structures used later for prompts, predictions, and report summaries.

```
# A string stores one inspection-style note.
note = "Mechanical component housing joint shows moderate cracking and minor leakage."

# A list stores several notes.
notes = [
    "Mechanical component housing joint shows moderate cracking and minor leakage.",
    "Pressure vessel appears stable with no visible distress.",
    "Cooling channel inlet has severe blockage after process debris."
]

# A dictionary stores structured information about one observation.
record = {"asset": "turbine blade surface", "defect": "cracking", "severity": "moderate"}

# A function performs one reusable task.
def short_preview(text, n_words=6):
    words = text.split()
    return " ".join(words[:n_words])

# A class can bundle a small report object and its method.
class MiniInspectionReport:
```

```
def __init__(self, asset, summary):
    self.asset = asset
    self.summary = summary

def one_line(self):
    return f"{self.asset}: {self.summary}"

report = MiniInspectionReport("Mechanical component A", short_preview(notes[0]))
print(record)
print(report.one_line())
```

Table 7. AI workflow vocabulary for absolute beginners. Source: Synthesized from Google Gemini API, OpenAI API, Hugging Face, and TensorFlow documentation.

| Term             | Definition                                                         | Example in this tutorial                                   |
|------------------|--------------------------------------------------------------------|------------------------------------------------------------|
| Prompt           | Instructions or context given to a generative model.               | "Summarize these inspection notes in three bullets."       |
| Response         | The output returned by a model or API.                             | A paragraph, caption, JSON object, or list of predictions. |
| Token            | A piece of text such as a word, punctuation mark, or subword unit. | "crack", "near", "joint".                                  |
| Text input       | A string or list of strings passed to a text workflow.             | Inspection comments.                                       |
| Image input      | An image, URL, or array passed to a vision workflow.               | A inspection photograph.                                   |
| Output object    | A structured Python result returned by a tool.                     | A list of dictionaries containing labels and scores.       |
| Local inference  | Running a model in the notebook environment.                       | A TensorFlow model trained on small classroom data.        |
| Hosted inference | Sending a request to a remote service.                             | OpenAI, Gemini, or Hugging Face Inference Providers.       |
| Proof of concept | A small demonstration used for learning and feasibility.           | A toy condition classifier trained in class.               |

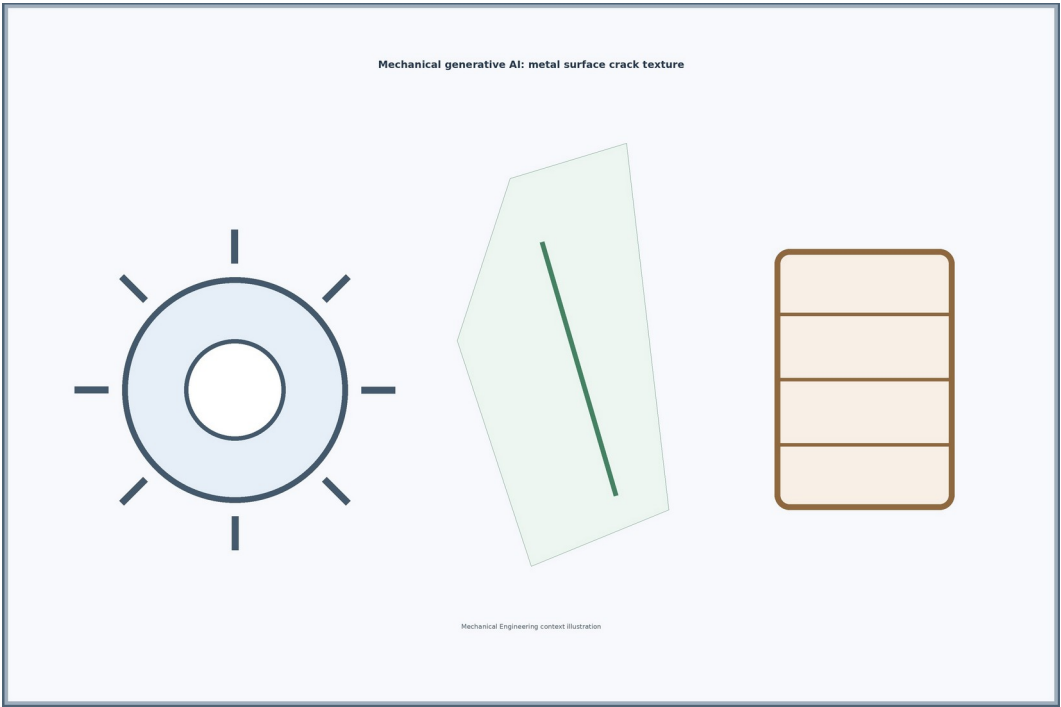


Figure 3. A complete cracked coated steel photograph. Image inputs in Mechanical Engineering may include machined surfaces, mechanical components, pressure vessels, cooling channels, or industrial safety scenes. Source: Marek Slusarczyk (2012), Wikimedia Commons, CC BY 3.0. Citation: Slusarczyk, 2012.

Table 8. Local workflows versus hosted inference workflows. Source: Synthesized from official OpenAI, Google Gemini, Hugging Face, and TensorFlow documentation.

| Workflow         | Where computation happens                     | Typical advantages                                                | Typical concerns                                               | Beginner example                                                    |
|------------------|-----------------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------|
| Local workflow   | In the notebook runtime or local computer.    | Transparent code, small experiments, no API bill for model calls. | Limited speed, memory, and model size.                         | Train a small TensorFlow classifier for inspection notes.           |
| Hosted inference | On a provider service reached through an API. | Access to powerful pretrained systems and multimodal tools.       | Credentials, cost, rate limits, privacy, and provider changes. | Send notes to OpenAI or Gemini for a draft summary.                 |
| Hybrid workflow  | Part local, part hosted.                      | Combines small predictions with language generation.              | More moving parts and more error handling.                     | Local classifier flags priority, API drafts a review-ready summary. |

**Section summary:** The examples use simple Python objects: strings, lists, dictionaries, arrays, tables, functions, classes, and methods. Students should learn to describe each workflow in terms of its input, processing step, and output before focusing on model details.

### 3. Part I: NLP Libraries and Hugging Face

Part I introduces beginner natural language processing and practical pretrained task interfaces. The sequence starts with spaCy and Natural Language Toolkit, or NLTK, because rule-based text processing is easy to inspect. It then moves to Hugging Face, where many text, image, audio, video, and multimodal tasks can be explored through consistent Python patterns.

*Table 9. Part I teaching map. Source: Synthesized from official spaCy, NLTK, and Hugging Face documentation.*

| Theme                            | Main tools                                                      | What students learn                                 | Mechanical Engineering application                                 |
|----------------------------------|-----------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------------------|
| spaCy and NLTK                   | Text cleaning, tokenization, stemming, EntityRuler-style rules. | How raw text becomes structured terms.              | Extract asset, defect, and severity terms from notes.              |
| Hugging Face                     | Transformers, Diffusers, task pages, pipeline-style patterns.   | How task interfaces map inputs to outputs.          | Generate text, caption images, process audio, inspect video tasks. |
| Introductory multimodal patterns | Image-to-text, image-text-to-text, any-to-any concepts.         | How mixed inputs can produce text or other outputs. | Link inspection images with inspection prompts and notes.          |

### 4. Introduction to spaCy and NLTK

Natural language processing, or NLP, is the study and engineering practice of working with human language using computation. For this tutorial, NLP means practical text workflows: cleaning notes, splitting notes into tokens, finding key terms, grouping severity words, and preparing text for a model.

spaCy is a Python library designed for production-style natural language processing, including tokenization, named entity recognition, and rule-based matching (spaCy, 2025a; spaCy, 2025b). NLTK is a long-standing Python toolkit for working with human language data and includes tokenization, stemming, tagging, and language-processing utilities (NLTK, 2025a; NLTK, 2025b).

*Table 10. spaCy and NLTK compared for beginners. Source: Synthesized from spaCy documentation and NLTK documentation.*

| Library       | Beginner strength                                                | Typical use in this tutorial                                                 | Mechanical Engineering interpretation                                       |
|---------------|------------------------------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| spaCy         | Fast pipeline objects, tokens, and rule-based entity extraction. | Use EntityRuler-style patterns to extract asset, defect, and severity terms. | Useful for turning inconsistent inspection notes into structured tables.    |
| NLTK          | Clear educational tools for tokenization and stemming.           | Use word tokenization and simple stemmers to study word forms.               | Useful for understanding how note words are split and normalized.           |
| Both together | Complementary learning value.                                    | Use NLTK for basic text mechanics and spaCy for practical extraction.        | A lightweight text workflow can support later reporting and classification. |

**Code Example 3. Install and import spaCy and NLTK**

Purpose: prepare beginner NLP libraries. Input: no engineering text yet. Output: imported tools and a blank English spaCy pipeline.

```
!pip install -q spacy nltk

import re
import nltk
import spacy
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

# NLTK data downloads are safe to repeat in Colab notebooks.
nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)

# A blank English spaCy pipeline is enough for rule-based examples.
nlp = spacy.blank("en")
print("spaCy and NLTK are ready.")
```

**Code Example 4. Basic cleaning, lowercasing, punctuation handling, and tokenization**

Purpose: show how an inspection note becomes a clean list of words. Output meaning: tokens are the pieces used for extraction and modeling.

```
raw_note = "Mount-3: MODERATE cracking near expansion joint; minor leakage observed."

# Lowercase makes matching easier when capitalization is inconsistent.
lower_note = raw_note.lower()

# Replace punctuation with spaces while keeping numbers and letters.
clean_note = re.sub(r"^[a-z0-9\s]", " ", lower_note)
clean_note = re.sub(r"\s+", " ", clean_note).strip()

# NLTK tokenization splits text into words and punctuation-aware pieces.
tokens = word_tokenize(clean_note)

print("Raw note:", raw_note)
print("Clean note:", clean_note)
print("Tokens:", tokens)
```

**Code Example 5. Simple stemming and token-length summaries**

Purpose: stemming maps related word forms toward a common root. Mechanical Engineering interpretation: cracking and cracked may become easier to group for simple keyword workflows.

```
stemmer = PorterStemmer()
notes = [
    "Moderate cracking near expansion joint.",
    "Severe corrosion on girder bearing plate.",
    "Minor spalling at material curb.",
    "No visible distress observed at cooling-channel inlet."
]

rows = []
for note in notes:
    tokens = word_tokenize(re.sub(r"^[a-z0-9\s]", " ", note.lower()))
    stems = [stemmer.stem(tok) for tok in tokens]
    rows.append({
        "note": note,
        "tokens": tokens,
        "stems": stems,
        "token_count": len(tokens),
        "avg_token_length": round(np.mean([len(t) for t in tokens]), 2)
    })

summary_table = pd.DataFrame(rows)
summary_table[["note", "token_count", "avg_token_length", "stems"]]
```

**Table 11. Text preprocessing steps in an inspection-note workflow. Source: Synthesized from NLTK tokenization documentation, NLTK Book Chapter 3, spaCy rule-based matching documentation, and spaCy EntityRuler documentation.**

| Step                 | Plain-language purpose                           | Example                                    | Caution                                  |
|----------------------|--------------------------------------------------|--------------------------------------------|------------------------------------------|
| Lowercasing          | Make matching case-insensitive.                  | Crack and crack become the same form.      | Acronyms may lose useful capitalization. |
| Punctuation handling | Remove characters that may interrupt matching.   | Mount-3 becomes mount 3.                   | Some symbols carry engineering meaning.  |
| Tokenization         | Split text into usable pieces.                   | "minor leakage" becomes minor and leakage. | Token rules differ across libraries.     |
| Stemming             | Reduce related words to shorter stems.           | cracking may become crack.                 | Stems are not always real words.         |
| Rule matching        | Find terms using dictionaries or patterns.       | defect: spalling.                          | Rules miss unfamiliar wording.           |
| Table output         | Store extracted information in rows and columns. | asset, defect, severity.                   | The table is only as good as the rules.  |

#### Code Example 6. spaCy tokenization and simple token statistics

Purpose: spaCy creates token objects with properties. Output meaning: each token can be tested and used in rules.

```
inspection_note = "Housing joint has severe leakage and cracking near the drain."
doc = nlp(inspection_note)

for token in doc:
    print(f"text={token.text:12s} is_punct={token.is_punct} length={len(token.text)}")

lengths = [len(token.text) for token in doc if not token.is_punct]
print("Average token length:", round(sum(lengths) / len(lengths), 2))
```

#### Code Example 7. Rule-based entity extraction with spaCy EntityRuler-style patterns

Purpose: extract asset type, defect type, and severity words with inspectable rules. Where used: note screening and searchable maintenance records.

```
import spacy

nlp = spacy.blank("en")
ruler = nlp.add_pipe("entity_ruler")

patterns = [
    {"label": "ASSET", "pattern": "turbine blade surface"},
    {"label": "ASSET", "pattern": "support mount"},
    {"label": "ASSET", "pattern": "cooling channel"},
    {"label": "ASSET", "pattern": "pressure vessel"},
    {"label": "DEFECT", "pattern": "cracking"},
    {"label": "DEFECT", "pattern": "spalling"},
    {"label": "DEFECT", "pattern": "corrosion"},
    {"label": "DEFECT", "pattern": "leakage"},
    {"label": "SEVERITY", "pattern": "minor"},
    {"label": "SEVERITY", "pattern": "moderate"},
    {"label": "SEVERITY", "pattern": "severe"}
]
ruler.add_patterns(patterns)

note = "Mechanical component housing shows severe cracking and minor leakage near the joint."
doc = nlp(note.lower())

for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

**Table 12. Example EntityRuler-style pattern groups for Mechanical Engineering notes. Source: Synthesized from spaCy rule-based matching and EntityRuler documentation.**

| Label  | Example terms                                                           | What the output means                                       | Possible next step                                    |
|--------|-------------------------------------------------------------------------|-------------------------------------------------------------|-------------------------------------------------------|
| ASSET  | turbine blade surface, support mount, cooling channel, pressure vessel. | The note mentions an mechanical systems asset or component. | Group notes by asset type.                            |
| DEFECT | cracking, spalling, corrosion, leakage, deflection.                     | The note contains a possible condition issue.               | Count defect mentions or production line to reviewer. |

| Label    | Example terms                              | What the output means                          | Possible next step                      |
|----------|--------------------------------------------|------------------------------------------------|-----------------------------------------|
| SEVERITY | minor, moderate, severe, urgent, critical. | The note includes a qualitative severity word. | Prioritize follow-up review.            |
| LOCATION | joint, bearing, drain, inlet, abutment.    | The note mentions a local component or area.   | Build more specific summary statements. |

### Code Example 8. Mini Mechanical Engineering text workflow over multiple notes

Purpose: produce a structured table from unstructured notes. Output: a table that can support sorting, counting, and later reporting.

```

notes = [
    "Mechanical component housing shows severe cracking near joint.",
    "Support mount has moderate corrosion at bearing plate.",
    "Cooling channel inlet has minor blockage and debris.",
    "Pressure vessel shows no visible distress.",
    "Material curb has minor spalling along edge."
]

records = []
for i, note in enumerate(notes, start=1):
    doc = nlp(note.lower())
    records.append({
        "id": i,
        "note": note,
        "assets": ", ".join([e.text for e in doc.ents if e.label_ == "ASSET"]),
        "defects": ", ".join([e.text for e in doc.ents if e.label_ == "DEFECT"]),
        "severity": ", ".join([e.text for e in doc.ents if e.label_ == "SEVERITY"])
    })

entity_table = pd.DataFrame(records)
entity_table

```

**Table 13. Mini workflow interpretation for inspection notes.** Source: Synthesized from NLTK and spaCy documentation and beginner Mechanical Engineering data-management practice.

| Workflow step    | Input                     | Output                                   | Mechanical Engineering use                    |
|------------------|---------------------------|------------------------------------------|-----------------------------------------------|
| Clean text       | Raw note.                 | Lowercase text with simpler punctuation. | Makes notes easier to compare.                |
| Tokenize         | Clean text.               | Words and punctuation-aware pieces.      | Supports counting and matching.               |
| Extract entities | Tokens plus rules.        | Asset, defect, and severity labels.      | Creates searchable records.                   |
| Build table      | Entity outputs.           | Rows with extracted fields.              | Supports dashboard or report draft.           |
| Review           | Table and original notes. | Human-checked interpretation.            | Prevents rule errors from becoming decisions. |

**Section summary:** spaCy and NLTK are useful first tools because they make text processing visible. Cleaning, tokenization, stemming, and rule-based extraction are not advanced generative AI, but they are essential preparation steps for practical engineering text workflows.

## 5. Theory and Background for Beginner NLP Workflows

The theory in this section is intentionally light. The goal is to understand how words become structured objects that a program can inspect. Advanced language models use complex learned representations, but the beginner path starts with tokens, vocabulary, rules, and simple counts.

**Table 14. Core beginner NLP concepts.** Source: Synthesized from NLTK Book Chapter 3, NLTK tokenization documentation, spaCy rule-based matching documentation, and spaCy EntityRuler documentation.

| Concept            | Plain-language explanation      | Simple mathematical or conceptual view | Engineering example                                    |
|--------------------|---------------------------------|----------------------------------------|--------------------------------------------------------|
| Text preprocessing | Preparing text before analysis. | Cleaned text = function(raw text).     | Remove inconsistent punctuation from inspection notes. |

| Concept            | Plain-language explanation                     | Simple mathematical or conceptual view | Engineering example                                      |
|--------------------|------------------------------------------------|----------------------------------------|----------------------------------------------------------|
| Token              | A piece of text used by a program.             | Sentence = [token_1, token_2, ...].    | "severe cracking" becomes severe and cracking.           |
| Vocabulary         | The set of known tokens.                       | V = unique tokens in the dataset.      | Known terms include crack, spall, leak, cooling channel. |
| Stemming           | Shortening related word forms.                 | stem(cracking) -> crack-like root.     | Group cracked, cracking, cracks.                         |
| Rule-based entity  | A span of text found by a rule.                | if phrase in dictionary, label it.     | Label cooling channel as ASSET.                          |
| Keyword extraction | Selecting words that carry meaning for a task. | keywords = important tokens.           | Find urgent, severe, corrosion.                          |

A simple rule-based NLP workflow can still be useful because many engineering notes are short and repetitive. For example, an inspection group may often write phrases such as "minor cracking," "moderate corrosion," or "severe blockage." A rule-based system can extract these phrases without training a machine learning model. This is valuable when data are limited, when the task is narrow, or when students need a transparent baseline.

The main limitation is that rules do exactly what they are told. A rule for "cracking" may miss "longitudinal crack," "hairline fracture," or a misspelled word. A rule-based table should therefore be treated as a starting point for review, not as a final engineering result.

**Table 15. Why simple rule-based NLP can be useful before machine learning.** Source: Synthesized from spaCy rule-based matching documentation and NLTK educational resources.

| Reason                 | Benefit                                       | Risk to manage                                    |
|------------------------|-----------------------------------------------|---------------------------------------------------|
| Transparency           | Students can inspect each rule and match.     | Transparent rules can still be incomplete.        |
| Small data             | Rules can work before labeled datasets exist. | Rules may not scale to diverse wording.           |
| Fast setup             | No training loop is required.                 | Poorly designed rules may produce false matches.  |
| Engineering vocabulary | Domain terms can be added directly.           | Synonyms and context are hard to handle manually. |
| Baseline value         | Creates a comparison point for later models.  | A baseline is not a validated model.              |

**Section summary:** Beginner NLP turns raw notes into tokens, vocabularies, patterns, and structured outputs. Rules are limited but valuable because they are inspectable, fast, and easy to connect to engineering vocabulary.

## 6. Introduction to Hugging Face

Hugging Face provides an ecosystem for finding models, datasets, educational resources, and task documentation. For beginners, the most important idea is the task interface: instead of building a model from scratch, students can choose a task such as text generation, image-to-text, text-to-speech, automatic speech recognition, image-to-image, text-to-image, video classification, or image-text-to-text. The Transformers pipeline interface is designed to make many inference tasks accessible through a compact Python call (Hugging Face, 2026a).

A pipeline-style interface hides preprocessing and model-call details. Students should still inspect inputs and outputs carefully. For Mechanical Engineering, Hugging Face tasks can support image captions, audio transcription, short text generation, image enhancement demonstrations, and video task exploration. Large generative media models may need specialized hardware or hosted inference, so the examples include graceful error handling and lightweight alternatives.

**Table 16. Hugging Face task families for beginner Mechanical Engineering workflows.** Source: Synthesized from Hugging Face Tasks pages for text generation, image-to-text, text-to-speech, automatic speech recognition, text-to-image, image-to-image, video classification, text-to-video, image-text-to-text, and any-to-any.

| Task family     | Input        | Output                         | Beginner Mechanical Engineering example |
|-----------------|--------------|--------------------------------|-----------------------------------------|
| Text generation | Text prompt. | Generated text continuation or | Draft a plain-language note             |

| Task family                  | Input                               | Output                                         | Beginner Mechanical Engineering example                 |
|------------------------------|-------------------------------------|------------------------------------------------|---------------------------------------------------------|
|                              |                                     | instruction response.                          | summary.                                                |
| Image-to-text                | Image.                              | Caption or optical-character-recognition text. | Caption a inspection image for an inspection log.       |
| Text-to-speech               | Text.                               | Audio waveform or file.                        | Convert a maintenance summary into spoken narration.    |
| Automatic speech recognition | Audio.                              | Text transcript.                               | Transcribe a field voice memo.                          |
| Text-to-image                | Text prompt.                        | Generated image.                               | Conceptual visual brainstorming, not evidence creation. |
| Image-to-image               | Image and optional text prompt.     | Transformed or enhanced image.                 | Explore super-resolution or image cleanup concepts.     |
| Video classification         | Video clip.                         | Class labels and scores.                       | Classify short activity clips in a classroom demo.      |
| Text-to-video                | Text prompt.                        | Generated video.                               | Understand emerging video generation interfaces.        |
| Image-text-to-text           | Image plus text prompt.             | Text answer.                                   | Ask a question about a inspection image.                |
| Any-to-any                   | Mixed text, image, video, or audio. | Mixed output such as text or audio.            | Study future multimodal engineering assistants.         |

### Code Example 9. Beginner Hugging Face text generation with a small model

Purpose: show input-output behavior for text generation. Input: a prompt. Output: generated text. Mechanical Engineering interpretation: useful for learning, but not a reliable inspection summary without review.

```
!pip install -q transformers torch

from transformers import pipeline

prompt = (
    "Summarize this inspection note in plain language: "
    "Mechanical component housing joint shows moderate cracking and minor leakage."
)

try:
    generator = pipeline("text-generation", model="distilgpt2")
    output = generator(
        prompt,
        max_new_tokens=40,
        num_return_sequences=1,
        truncation=True
    )
    print(output[0]["generated_text"])
except Exception as err:
    print("Text generation example could not run in this runtime.")
    print("Reason:", err)
```

### Code Example 10. Image-to-text captioning pattern with graceful handling

Purpose: demonstrate the image-to-text task. For Mechanical Engineering, replace the example image with an asset image only when privacy and data rules allow it.

```
!pip install -q transformers pillow requests torch

from transformers import pipeline
from PIL import Image
import requests
from io import BytesIO

image_url = "https://huggingface.co/datasets/Narsil/image_dummy/resolve/main/parrots.png"

try:
    captioner = pipeline("image-to-text", model="Salesforce/blip-image-captioning-base")
    image = Image.open(BytesIO(requests.get(image_url, timeout=10).content)).convert("RGB")
    caption = captioner(image)
    print(caption)
except Exception as err:
    print("Image-to-text example could not run in this runtime.")
    print("Reason:", err)
```

**Table 17. Input and output meaning for selected Hugging Face tasks. Source: Synthesized from Hugging Face task documentation and Transformers pipeline documentation.**

| Task                         | Input meaning                              | Output meaning                | How students should inspect it                                       |
|------------------------------|--------------------------------------------|-------------------------------|----------------------------------------------------------------------|
| Text generation              | A prompt or beginning of text.             | A generated text string.      | Check whether the response follows instructions and preserves facts. |
| Image-to-text                | An image object, image URL, or image file. | A caption or recognized text. | Compare caption against the image.                                   |
| Text-to-speech               | Text to be spoken.                         | Audio output.                 | Listen for clarity and correctness.                                  |
| Automatic speech recognition | Audio recording.                           | Transcript string.            | Compare transcript with the recording.                               |
| Image-to-image               | Starting image and optional prompt.        | Transformed image.            | Check whether important engineering details were changed.            |
| Video classification         | Short video.                               | Labels and scores.            | Check whether labels match visible activity.                         |
| Image-text-to-text           | Image plus text question or instruction.   | Text answer.                  | Verify answer against image evidence.                                |

#### Code Example 11. Hosted text-to-speech request pattern with missing-token handling

Purpose: show the request-response idea for an audio generation task while avoiding a crash when credentials are missing.

```
# This example uses a hosted request pattern. It does not run without a token.
# In Colab, store the token as an environment variable named HF_TOKEN.

import os
import requests

HF_TOKEN = os.getenv("HF_TOKEN")
API_URL = "https://production.liner.huggingface.co/hf-inference/models/facebook/mms-tts-eng"

payload = {"inputs": "Mechanical component housing inspection summary: moderate cracking observed."}

if not HF_TOKEN:
    print("HF_TOKEN is not set. Skipping hosted text-to-speech call.")
else:
    headers = {"Authorization": f"Bearer {HF_TOKEN}"}
    response = requests.post(API_URL, headers=headers, json=payload, timeout=60)
    print("Status code:", response.status_code)
    print("Returned bytes:", len(response.content))
```

#### Code Example 12. Automatic speech recognition pattern for a field audio memo

Purpose: introduce speech-to-text. Mechanical Engineering interpretation: field voice memos can become searchable text, but transcripts need review.

```
!pip install -q transformers torch

from transformers import pipeline

try:
    transcriber = pipeline(
        "automatic-speech-recognition",
        model="openai/whisper-tiny.en"
    )
    # Replace sample.wav with a short classroom audio file.
    # result = transcriber("sample.wav")
    # print(result["text"])
    print("Transcriber created. Provide a short WAV file to run the example.")
except Exception as err:
    print("ASR example could not initialize in this runtime.")
    print("Reason:", err)
```

#### Code Example 13. Text-to-image and image-to-image patterns for optional GPU runtimes

Purpose: show that text-to-image and image-to-image tasks require more resources than text examples.

Engineering caution: generated images must never be treated as field evidence.

```
# Generative image examples can be memory intensive.
# Treat this as an optional GPU demonstration, not a required CPU exercise.

!pip install -q diffusers transformers accelerate safetensors

import torch
from diffusers import AutoPipelineForText2Image

prompt = "simple line drawing of a mechanical component inspection checklist, clean educational style"

if not torch.cuda.is_available():
    print("GPU is not available. Skipping text-to-image generation.")
else:
    pipe = AutoPipelineForText2Image.from_pretrained(
        "stabilityai/sd-xl-turbo",
        torch_dtype=torch.float16,
        variant="fp16"
    ).to("cuda")
    image = pipe(prompt=prompt, num_inference_steps=1, guidance_scale=0.0).images[0]
    image.save("generated_checklist_concept.png")
    print("Saved generated_checklist_concept.png")
```

#### Code Example 14. Video classification pattern

Purpose: show that video tasks use the same general input-output pattern: a media input becomes labels and scores.

```
!pip install -q transformers torch torchvision

from transformers import pipeline

try:
    video_pipe = pipeline(
        task="video-classification",
        model="nateraw/videomae-base-finetuned-ucf101-subset"
    )
    print("Video classifier created.")
    print("Provide a short video path or URL to run prediction.")
except Exception as err:
    print("Video classification example could not initialize in this runtime.")
    print("Reason:", err)
```

**Table 18. Practical access patterns for Hugging Face workflows.** Source: Synthesized from Hugging Face Transformers pipeline documentation, Hugging Face task pages, and Hugging Face Inference Providers pricing documentation.

| Access pattern           | What it means                                                   | Good for beginners                          | Caution                                                              |
|--------------------------|-----------------------------------------------------------------|---------------------------------------------|----------------------------------------------------------------------|
| Local pipeline           | Download a compatible model and run it in the notebook.         | Small text and small classification tasks.  | Large models may exceed memory or take a long time.                  |
| Diffusers local pipeline | Run image generation or image transformation locally.           | Optional GPU demonstrations.                | CPU execution can be impractical for classroom time.                 |
| Hosted inference request | Send a request to a remote provider through an API.             | Trying larger tasks without local hardware. | Requires credentials, may involve pricing, and may have rate limits. |
| Task page exploration    | Study task inputs, outputs, examples, and compatible libraries. | Understanding capabilities before coding.   | Demo widgets are not a substitute for validation.                    |

**Section summary:** Hugging Face is useful for learning task interfaces. Students should focus on the mapping from input to output, the returned object, runtime requirements, and the difference between local execution and hosted inference.

## 7. Hugging Face Task-Interface Understanding

A pipeline-style interface is a wrapper that performs several steps for a task: prepare the input, send it through a model, and return a readable output object. For example, a text classification pipeline can tokenize a string, run a

model, and return a label and score. The student does not need to write the tokenizer and model-call details separately for a first demonstration (Hugging Face, 2026a).

**Table 19. Pipeline-style interface anatomy.** Source: Synthesized from Hugging Face Transformers pipeline documentation and task pages.

| Piece           | Plain-language meaning                | Example                                                               |
|-----------------|---------------------------------------|-----------------------------------------------------------------------|
| Task name       | The type of problem to run.           | "text-generation", "image-to-text", "automatic-speech-recognition".   |
| Model name      | The specific pretrained model to use. | "distilgpt2" or "Salesforce/blip-image-captioning-base".              |
| Input object    | The data passed into the task.        | A string, image, audio path, video path, or mixed message.            |
| Parameters      | Settings controlling output behavior. | max_new_tokens, temperature, top_k, or image strength.                |
| Output object   | The returned Python result.           | A string, list of dictionaries, image object, audio bytes, or labels. |
| Inspection step | Human review of the output.           | Compare caption with image or transcript with audio.                  |

### Code Example 15. Comparing input-output behavior across task types

Purpose: inspect task interfaces as data. This is a beginner-friendly way to compare workflows before running models.

```
examples = [
    {
        "task": "text-generation",
        "input_type": "text prompt",
        "output_type": "generated text",
        "mechanical_example": "Draft a maintenance summary."
    },
    {
        "task": "image-to-text",
        "input_type": "image",
        "output_type": "caption text",
        "mechanical_example": "Caption a inspection photograph."
    },
    {
        "task": "automatic-speech-recognition",
        "input_type": "audio",
        "output_type": "transcript text",
        "mechanical_example": "Transcribe a field memo."
    },
    {
        "task": "video-classification",
        "input_type": "video",
        "output_type": "labels and scores",
        "mechanical_example": "Classify activity in a short clip."
    }
]

pd.DataFrame(examples)
```

**Table 20. Output objects differ across modalities.** Source: Synthesized from Hugging Face task pages and Transformers pipeline documentation.

| Task                 | Typical Python output                       | Beginner interpretation                             |
|----------------------|---------------------------------------------|-----------------------------------------------------|
| Text generation      | [{"generated_text": "..."}] or text string. | The model produced words. Read them critically.     |
| Image-to-text        | [{"generated_text": "..."}].                | The model described the image or recognized text.   |
| Text-to-speech       | Audio bytes or an audio array.              | The model produced sound from text.                 |
| Text-to-image        | PIL image object or image file.             | The model produced an image from a prompt.          |
| Image-to-image       | PIL image object or image file.             | The model transformed the starting image.           |
| Video classification | [{"label": "...", "score": 0.90}, ...].     | The model produced class probabilities.             |
| Image-text-to-text   | Generated text answer.                      | The model used an image and a text prompt together. |

For students, the safest way to learn task interfaces is to print the returned object, identify the fields, and compare the result with the input. For example, a caption should be compared with the actual image. A transcript should be compared with the audio. A generated maintenance statement should be compared with the original inspection notes.

**Section summary:** A task interface is a predictable input-output pattern. The same student habit applies to all modalities: identify the input, run the task, inspect the returned object, and verify the output against the original evidence.

## 8. Introductory Hugging Face Multimodal Task Patterns

A multimodal model works with more than one type of input or output. A simple example is image-to-text, where an image becomes a caption. A more flexible example is image-text-to-text, where the model receives both an image and a text prompt, then returns a text answer. Hugging Face describes image-text-to-text systems as vision-language models that take an image and text prompt and output text (Hugging Face, 2026i).

### A 4-dimensional embedding

|               |     |      |      |     |
|---------------|-----|------|------|-----|
| <b>cat</b> => | 1.2 | -0.1 | 4.3  | 3.2 |
| <b>mat</b> => | 0.4 | 2.5  | -0.9 | 0.5 |
| <b>on</b> =>  | 2.1 | 0.3  | 0.1  | 0.4 |
| ...           | ... | ...  | ...  | ... |

*Figure 4. A TensorFlow educational visualization of word embeddings. Embeddings help students understand that text can be represented numerically before it is combined with other information. Source: TensorFlow Text guide (2024), TensorFlow documentation. Citation: TensorFlow Text, 2024.*

#### Code Example 16. Image-text-to-text message pattern

Purpose: show how an image and text prompt can be bundled together. Output: a structured message object ready for a multimodal interface.

```
# This example shows the message structure used by many vision-language tasks.
# Running a vision-language model may require a GPU or a hosted endpoint.

messages = [
    {
        "role": "user",
        "content": [
            {"type": "image", "image": "site_photo.jpg"},
            {"type": "text", "text": "Describe visible defects in one cautious sentence."}
        ]
    }
]

print(messages)
```

**Table 21. Image-text pairing examples in Mechanical Engineering. Source: Synthesized from Hugging Face image-to-text and image-text-to-text task documentation and Mechanical Engineering inspection examples.**

| Image input                              | Text prompt                                    | Expected output type              | Review question                            |
|------------------------------------------|------------------------------------------------|-----------------------------------|--------------------------------------------|
| Machined surface photograph.             | Describe visible surface distress cautiously.  | Caption or condition description. | Does the caption match the image?          |
| Mechanical component bearing photograph. | List visible components and possible concerns. | Short observation list.           | Did the model confuse components?          |
| Cooling channel inlet image.             | Is blockage visible? Answer with uncertainty.  | Text answer.                      | Is the answer supported by the image?      |
| Manufacturing cell photo.                | Create a neutral record-keeping caption.       | Plain-language caption.           | Does the caption avoid unsupported claims? |

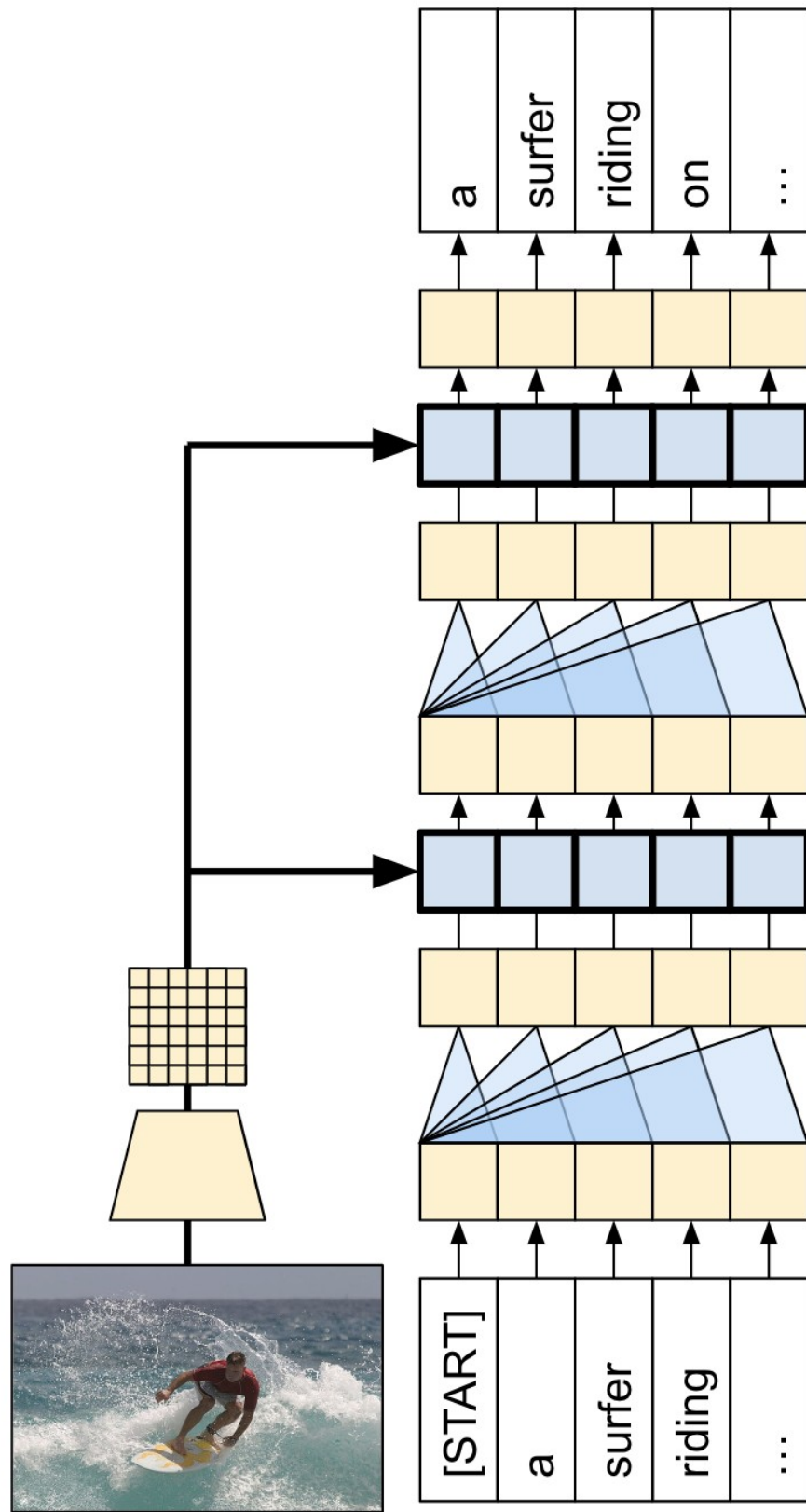


Figure 5. A complete TensorFlow image captioning architecture diagram. It shows how image features and text tokens can interact in an image-to-text workflow. Source: TensorFlow Image Captioning tutorial (2024), TensorFlow documentation. Citation: TensorFlow, 2024e.

Any-to-any style interfaces are an emerging direction in which a model may take combinations of text, image, video, and audio and may respond through text, audio, actions, predictions, or generated media. Hugging Face describes any-to-any models as systems that can take multimodal inputs and generate different types of outputs (Hugging Face, 2026j). For beginners, the goal is only to understand the interface concept, not to train or deploy a large system.

Table 22. Any-to-any style interface examples for future Mechanical Engineering workflows. Source: Synthesized from Hugging Face any-to-any and image-text-to-text documentation, OpenAI multimodal documentation, and Google Gemini API documentation.

| Input combination       | Possible output              | Future engineering use                           | Beginner caution                       |
|-------------------------|------------------------------|--------------------------------------------------|----------------------------------------|
| Image plus text prompt. | Text answer.                 | Ask questions about an asset photo.              | Answers need image verification.       |
| Video plus text prompt. | Caption or activity summary. | Summarize a short site clip.                     | Video context can be missed.           |
| Audio plus text prompt. | Transcript and summary.      | Summarize voice notes from field work.           | Transcription errors can propagate.    |
| Table plus prompt.      | Narrative summary.           | Explain dashboard trends.                        | The model must not invent numbers.     |
| Image, note, and table. | Compact report draft.        | Create a review-ready asset condition paragraph. | Professional review remains necessary. |

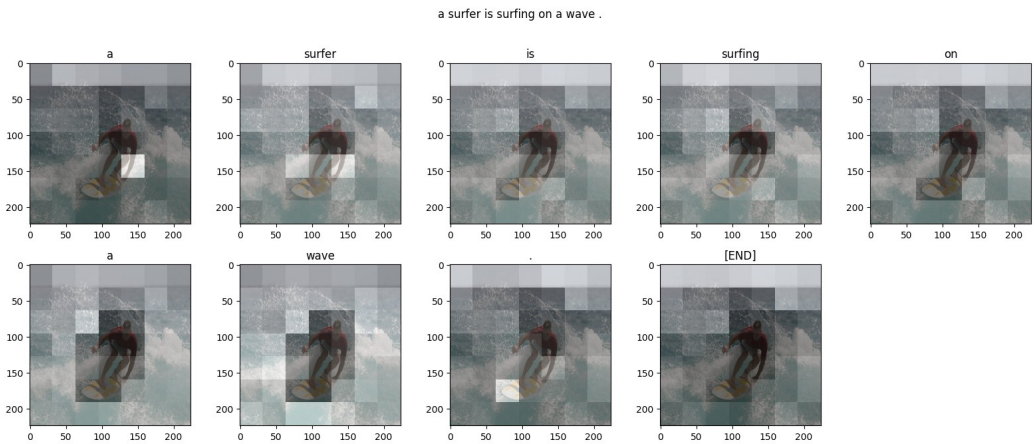


Figure 6. A complete TensorFlow image captioning example showing visual attention while words are generated. The example helps students understand that image regions and text tokens can interact. Source: TensorFlow Image Captioning tutorial (2024), TensorFlow documentation. Citation: TensorFlow, 2024e.

**Section summary:** Multimodal workflows combine data types. The beginner skill is to identify each input type, understand the output object, and verify that generated text is supported by the image, audio, video, or table evidence.

## 9. Part II: APIs and Simple Mechanical Engineering AI Workflows

Part II introduces application programming interfaces, or APIs, and small local machine learning workflows. An API is a way for one program to send a request to another service and receive a response. In AI work, a request often contains a prompt and settings, while the response contains generated text, a caption, a transcript, a structured object, or other output.

Table 23. Part II teaching map. Source: Synthesized from official OpenAI, Google Gemini, TensorFlow, spaCy, NLTK, and Hugging Face documentation.

| Theme                         | Main tools                                                  | What students learn                                                    | Mechanical Engineering application                            |
|-------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------|---------------------------------------------------------------|
| Google and OpenAI APIs        | Google GenAI SDK, OpenAI Python SDK, environment variables. | How requests, responses, API keys, and costs work at a beginner level. | Generate review-ready summaries from inspection notes.        |
| Simple AI and ML applications | TensorFlow, Keras, pandas, spaCy/NLTK preprocessing.        | How lightweight models classify text, images, and tables.              | Inspection note classification and condition screening demos. |
| Workflow integration          | Local preprocessing plus local                              | How separate pieces form an                                            | Raw notes and image-screening                                 |

| Theme | Main tools                   | What students learn      | Mechanical Engineering application    |
|-------|------------------------------|--------------------------|---------------------------------------|
|       | prediction plus API summary. | end-to-end mini project. | scores become a compact report draft. |

## 10. Google and OpenAI APIs for Beginner Mechanical Engineering Workflows

OpenAI and Google both provide APIs that can receive text prompts and return generated text. The exact models, prices, and rate limits change over time, so students should always verify current access and pricing from official provider pages before using paid services. This tutorial uses safe code patterns: environment variables for API keys, missing-key checks, simple prompts, and printed response inspection.

**Table 24. Basic request-response structure for beginner API calls. Source: Synthesized from OpenAI API libraries documentation and Google Gemini API quickstart documentation.**

| Piece              | Plain-language meaning                                | Example                                                      |
|--------------------|-------------------------------------------------------|--------------------------------------------------------------|
| API key            | A secret credential used to authenticate the request. | OPENAI_API_KEY or GEMINI_API_KEY in an environment variable. |
| Client object      | A Python object that knows how to send requests.      | OpenAI() or genai.Client().                                  |
| Model name         | The model selected for the task.                      | Provider-specific model identifier.                          |
| Prompt or contents | The instructions and data sent to the model.          | Inspection notes plus a summarization request.               |
| Response object    | The returned result from the provider.                | Text, metadata, and possibly structured fields.              |
| Inspection step    | Human review of the output.                           | Check that the summary is faithful to original notes.        |

**Table 25. Current official beginner access and billing notes to verify before use. Source: Synthesized from current official provider documentation accessed 2026-04-24.**

| Provider                      | Official beginner access statement                                                                                                       | Cost or limit point to verify                                                                                                                                                                                                 | Primary official sources                                                                                        |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| OpenAI API                    | The official quickstart describes creating an API key and using official SDKs. The OpenAI SDK reads OPENAI_API_KEY from the environment. | Usage is billed according to official pricing. OpenAI Help states new accounts are enrolled in prepaid billing, purchased credits can be used for API usage, and any free credits on an account are used before paid credits. | OpenAI API quickstart, OpenAI libraries documentation, OpenAI API pricing, OpenAI prepaid billing help article. |
| Google Gemini API             | The official quickstart states that using the Gemini API requires an API key and that one can be created for free to get started.        | Google states new accounts begin on the Free Tier for certain models up to free-tier rate limits, and moving to a paid tier requires setting up billing in AI Studio. Free-tier availability and limits can vary.             | Gemini API quickstart, API keys, billing, pricing, rate limits, and troubleshooting documentation.              |
| Hugging Face hosted inference | Hugging Face Inference Providers provide access to models through integrated client SDKs and serverless providers.                       | Inference Providers documentation describes centralized pay-as-you-go pricing with no markup from Hugging Face. Students should check model and provider availability before use.                                             | Hugging Face Inference Providers and pricing documentation.                                                     |

### Code Example 17. OpenAI API text request with missing-key handling

Purpose: send a simple text request. Input: an inspection-note prompt. Output: a text response. The missing-key check keeps the notebook from crashing in classrooms.

```
!pip install -q openai
```

```

import os
from openai import OpenAI

prompt = """
You are helping a mechanical engineering student.
Summarize this inspection note in two cautious bullet points:
Mechanical component housing joint shows moderate cracking and minor leakage.
"""

api_key = os.getenv("OPENAI_API_KEY")

if not api_key:
    print("OPENAI_API_KEY is not set. Skipping OpenAI API call.")
else:
    client = OpenAI()
    response = client.responses.create(
        model=os.getenv("OPENAI_MODEL", "gpt-5.4-mini"),
        input=prompt
    )
    print(response.output_text)

```

### Code Example 18. Google Gemini API text request with missing-key handling

Purpose: show the same request-response idea with Google Gemini. Input: a prompt. Output: response.text.

```

!pip install -q -U google-gemai

import os
from google import genai

prompt = """
Summarize this mechanical engineering inspection note in two cautious bullet points:
Mechanical component housing joint shows moderate cracking and minor leakage.
"""

api_key = os.getenv("GEMINI_API_KEY")

if not api_key:
    print("GEMINI_API_KEY is not set. Skipping Gemini API call.")
else:
    client = genai.Client()
    response = client.models.generate_content(
        model=os.getenv("GEMINI_MODEL", "gemini-3-flash-preview"),
        contents=prompt
    )
    print(response.text)

```

### Code Example 19. Comparing OpenAI and Gemini on the same small task

Purpose: separate prompt construction from provider calls. Students can evaluate outputs consistently across providers.

```

inspection_notes = [
    "Mechanical component housing joint shows moderate cracking and minor leakage.",
    "Cooling channel inlet has severe blockage after process debris.",
    "Pressure vessel appears stable with no visible distress."
]

prompt = """
Create a concise maintenance review summary from these notes.
Use cautious wording and do not invent measurements.
Notes:
""" + "\n".join(f"- {n}" for n in inspection_notes)

print(prompt)

# Send this same prompt to one provider at a time.
# Compare whether each response preserves the same facts.

```

**Table 26. OpenAI and Gemini beginner workflow comparison.** Source: Synthesized from OpenAI libraries documentation and Google Gemini API quickstart documentation.

| Step               | OpenAI pattern                             | Gemini pattern                                    | Common beginner principle                     |
|--------------------|--------------------------------------------|---------------------------------------------------|-----------------------------------------------|
| Install library    | <code>pip install openai.</code>           | <code>pip install -U google-gemai.</code>         | Use official SDKs when available.             |
| Store key          | <code>OPENAI_API_KEY.</code>               | <code>GEMINI_API_KEY.</code>                      | Do not hard-code secrets in shared notebooks. |
| Create client      | <code>client = OpenAI().</code>            | <code>client = gemai.Client().</code>             | The client sends requests.                    |
| Send text          | <code>client.responses.create(...).</code> | <code>client.models.generate_content(...).</code> | Prompt text is the main input.                |
| Read output        | <code>response.output_text.</code>         | <code>response.text.</code>                       | Always inspect returned text.                 |
| Handle missing key | Skip call and print message.               | Skip call and print message.                      | Examples should fail gracefully.              |

A beginner API workflow should never assume that a key exists, a model is available, or a service is free. The code should check credentials, print helpful messages, and keep raw input data visible so that outputs can be checked. For student projects, it is also good practice to save prompts and model outputs in a small table for comparison.

**Section summary:** APIs are request-response systems. Good beginner API code stores keys safely, handles missing keys gracefully, uses small prompts, prints returned outputs, and checks provider access and pricing from official documentation before use.

## 11. Simple Mechanical Engineering Applications of AI and Machine Learning

This section introduces local TensorFlow workflows that support practical AI applications when students need a small model rather than a hosted generative response. The examples are intentionally lightweight. They are designed to teach concepts such as features, labels, training, prediction, and interpretation. They are not certified engineering tools.

*Table 27. Simple Mechanical Engineering applications and matching beginner tools. Source: Synthesized from TensorFlow text classification, image classification, and structured data tutorials, plus OpenAI and Gemini text-generation documentation.*

| Application                         | Input                     | Local tool                                     | Generated or predicted output  | Review step                                  |
|-------------------------------------|---------------------------|------------------------------------------------|--------------------------------|----------------------------------------------|
| Inspection-note text classification | Short notes.              | TextVectorization plus small TensorFlow model. | Priority class or probability. | Compare against human labels.                |
| Simple image condition screening    | Toy or controlled images. | Small convolutional neural network.            | Condition score or class.      | Inspect false positives and false negatives. |
| Small tabular workflow              | Feature table.            | Dense neural network or simple baseline.       | Risk or priority score.        | Check feature units and data quality.        |
| Report-style summary                | Model outputs plus notes. | API text generation.                           | Draft narrative.               | Verify all facts against inputs.             |

### Code Example 20. Lightweight TensorFlow inspection-note text classification

Purpose: classify short notes. Input: text strings. Output: probability of priority review. Mechanical Engineering interpretation: a classroom model can teach workflow mechanics, but real labels require expert quality control.

```
!pip install -q tensorflow pandas

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np

texts = np.array([
    "minor cracking near joint",
    "no visible distress observed",
    "severe corrosion at bearing",
    "moderate leakage at housing drain",
    "surface appears stable",
    "urgent blockage at cooling-channel inlet",
    "minor spalling on curb",
    "severe cracking and leakage"
])
```

```

# 0 = routine review, 1 = priority review
labels = np.array([0, 0, 1, 1, 0, 1, 0, 1])

vectorizer = layers.TextVectorization(
    max_tokens=100,
    output_mode="int",
    output_sequence_length=8
)
vectorizer.adapt(texts)

model = keras.Sequential([
    keras.Input(shape=(1,)), dtype=tf.string),
    vectorizer,
    layers.Embedding(input_dim=100, output_dim=8),
    layers.GlobalAveragePooling1D(),
    layers.Dense(8, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])

model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(texts, labels, epochs=30, verbose=0)

new_notes = np.array([
    "moderate corrosion near bearing",
    "no distress at pressure vessel"
])
probs = model.predict(new_notes, verbose=0).ravel()

for note, prob in zip(new_notes, probs):
    print(f"{note:35s} priority probability = {prob:.3f}")

```

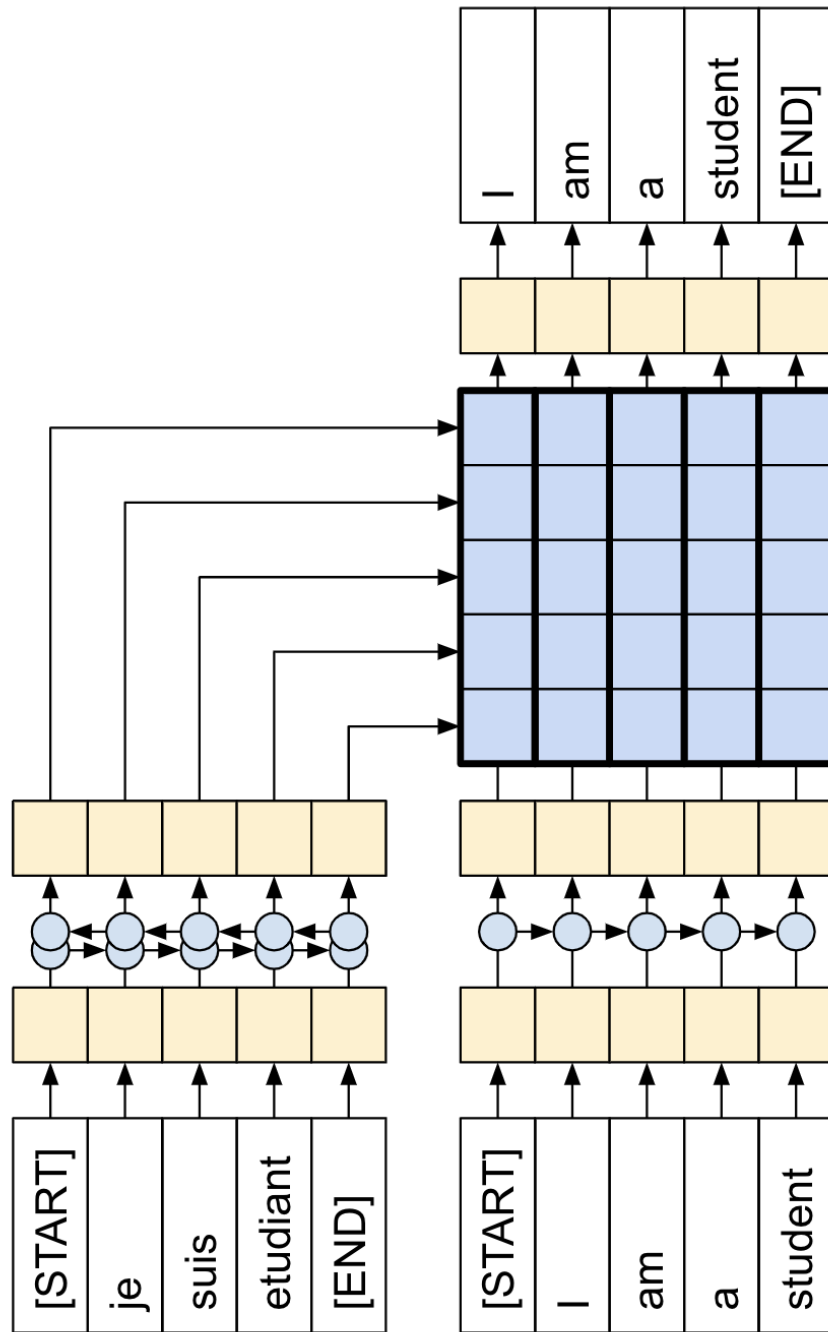


Figure 7. A complete TensorFlow educational diagram showing attention in a sequence model. It helps students understand why modern text models can focus on relevant words instead of treating every position equally. Source: TensorFlow Transformer tutorial (2024), TensorFlow documentation. Citation: TensorFlow, 2024d.

#### Code Example 21. Simple image condition screening with toy arrays

Purpose: demonstrate image classification with tiny arrays. Output: a condition score. Engineering caution: toy arrays are not real inspection data.

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

rng = np.random.default_rng(7)

def make_toy_image(has_line, size=24):
```

```

img = rng.normal(0.05, 0.03, (size, size, 1)).astype("float32")
if has_line:
    row = rng.integers(6, size - 6)
    for col in range(4, size - 4):
        img[row + rng.integers(-1, 2), col, 0] = 1.0
    return np.clip(img, 0, 1)

images = np.stack([make_toy_image(i % 2 == 1) for i in range(200)])
labels = np.array([i % 2 for i in range(200)])

cnn = keras.Sequential([
    keras.Input(shape=(24, 24, 1)),
    layers.Conv2D(8, 3, activation="relu", padding="same"),
    layers.MaxPooling2D(),
    layers.Conv2D(16, 3, activation="relu", padding="same"),
    layers.GlobalAveragePooling2D(),
    layers.Dense(1, activation="sigmoid")
])

cnn.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
cnn.fit(images, labels, epochs=8, batch_size=32, verbose=0)

score = cnn.predict(images[:4], verbose=0).ravel()
print("First four condition scores:", np.round(score, 3))

```

### Code Example 22. Small tabular workflow for asset-priority scoring

Purpose: show a small tabular model. Input: numerical features. Output: a priority probability. Mechanical Engineering interpretation: features and labels must be meaningful and verified.

```

import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

asset_data = pd.DataFrame({
    "age_years": [5, 12, 18, 30, 45, 8, 22, 38],
    "traffic_level": [1, 2, 2, 3, 3, 1, 2, 3],
    "defect_count": [0, 1, 2, 4, 5, 0, 3, 4],
    "priority": [0, 0, 0, 1, 1, 0, 1, 1]
})

x = asset_data[["age_years", "traffic_level", "defect_count"]].astype("float32")
y = asset_data["priority"].astype("float32")

normalizer = layers.Normalization()
normalizer.adapt(x)

tabular_model = keras.Sequential([
    keras.Input(shape=(3,)),
    normalizer,
    layers.Dense(8, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])

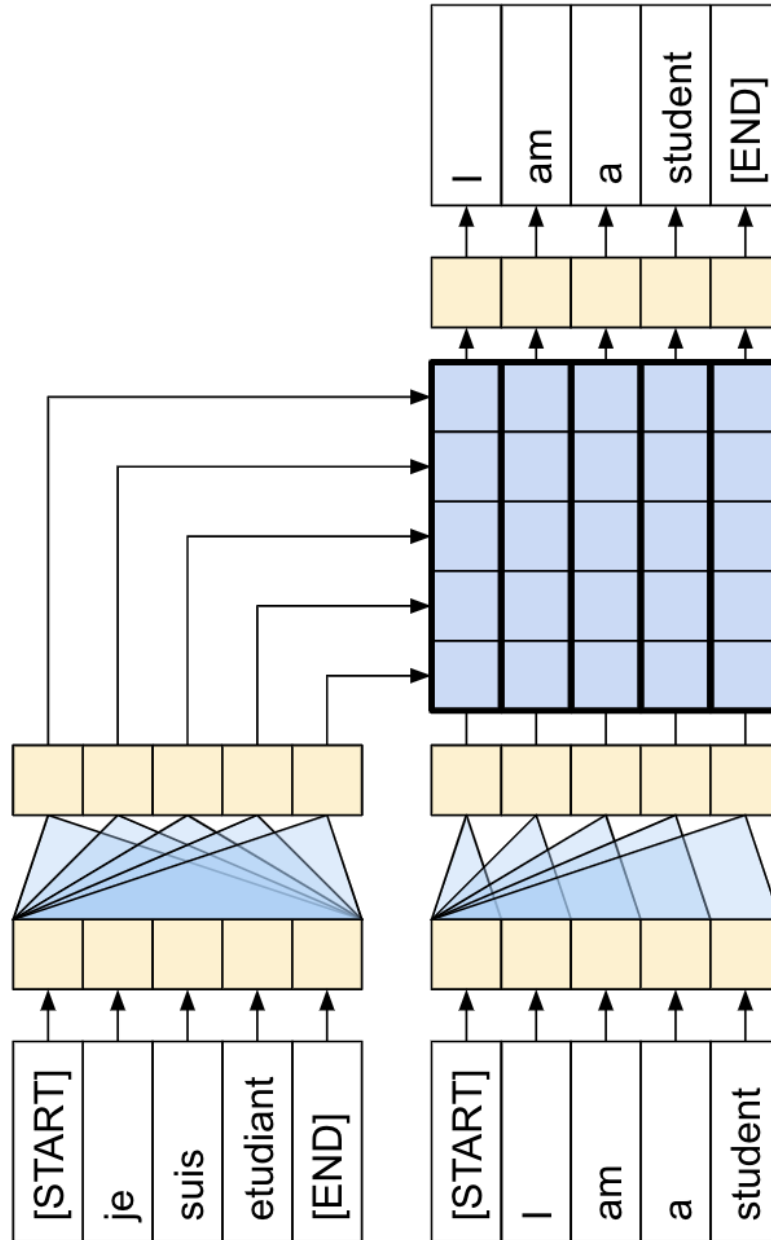
tabular_model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
tabular_model.fit(x, y, epochs=80, verbose=0)

new_asset = np.array([[25, 3, 2]], dtype="float32")
print("Priority probability:", float(tabular_model.predict(new_asset, verbose=0)[0, 0]))

```

**Table 28. Interpreting model outputs in beginner Mechanical Engineering workflows. Source: Synthesized from TensorFlow tutorials and official API documentation for generative outputs.**

| Output type       | What it means                                | What it does not mean                                                        | Student check                         |
|-------------------|----------------------------------------------|------------------------------------------------------------------------------|---------------------------------------|
| Probability       | The model score between 0 and 1 for a class. | It is not a measured probability of failure unless calibrated and validated. | Compare with labels and examples.     |
| Label             | The model class with the highest score.      | It is not an official inspection decision.                                   | Review original input.                |
| Generated summary | A text draft based on the prompt and data.   | It is not automatically true.                                                | Trace each sentence to source inputs. |
| Caption           | A model description of an image.             | It is not a certified condition assessment.                                  | Compare with visible evidence.        |



*Figure 8. A complete TensorFlow educational diagram of a one-layer transformer processing words. While this tutorial focuses on practical interfaces, this visual motivates why modern text tools can represent word interactions. Source: TensorFlow Transformer tutorial (2024), TensorFlow documentation. Citation: TensorFlow, 2024d.*

**Section summary:** Small TensorFlow models can classify notes, images, and tabular records. These workflows help students understand training and prediction, while hosted generative APIs can turn verified outputs into readable summaries.

## 12. Beginner Workflow Integration

Workflow integration means connecting several small steps into a useful process. A realistic beginner project might clean notes, extract defect words, classify priority with a local model, then send a compact verified

summary to an API for plain-language drafting. Integration matters because engineering tasks rarely use only one kind of data or one tool.

**Table 29. End-to-end Mechanical Engineering AI mini-project flow.** Source: Synthesized from spaCy, NLTK, TensorFlow, OpenAI, and Google Gemini documentation.

| Step       | Input                                       | Tool                             | Output                          | Review focus                   |
|------------|---------------------------------------------|----------------------------------|---------------------------------|--------------------------------|
| Collect    | Short inspection notes.                     | Python list or table.            | Raw notes.                      | Are notes complete enough?     |
| Preprocess | Raw notes.                                  | NLTK or spaCy.                   | Tokens, cleaned text, entities. | Were important terms retained? |
| Predict    | Cleaned notes.                              | Small TensorFlow text model.     | Priority probabilities.         | Do scores make sense?          |
| Summarize  | Original notes plus verified model outputs. | OpenAI or Gemini API.            | Plain-language draft.           | Are facts faithful to inputs?  |
| Archive    | Inputs, outputs, and review notes.          | Pandas table or notebook export. | Reproducible record.            | Can another person audit it?   |

### Code Example 23. Combining NLP preprocessing with a lightweight TensorFlow model

Purpose: turn a raw note into a clean text string and a model output. Mechanical Engineering interpretation: the output can support triage but must be reviewed.

```
# Reuse the trained text model from Code Example 20.
# This function keeps preprocessing and prediction together.

def predict_note_priority(note_text):
    clean = re.sub(r"[^a-z0-9\s]", " ", note_text.lower())
    clean = re.sub(r"\s+", " ", clean).strip()
    probability = float(model.predict(np.array([clean]), verbose=0)[0, 0])
    label = "priority review" if probability >= 0.5 else "routine review"
    return {"clean_text": clean, "probability": probability, "label": label}

result = predict_note_priority("Severe leakage and cracking at turbine blade surface joint.")
print(result)
```

### Code Example 24. Combining a local model with an API prompt

Purpose: separate verified local outputs from generated narrative. This helps prevent the API from inventing facts.

```
# This code builds a verified prompt from local outputs.
# It does not call an API unless you add the provider-specific code from Section 10.

note = "Severe leakage and cracking at turbine blade surface joint."
local_result = predict_note_priority(note)

summary_prompt = f"""
Create a concise maintenance review draft from the information below.
Do not invent measurements, locations, or dates.
Original note: {note}
Local model label: {local_result['label']}
Local model probability: {local_result['probability']:.3f}
Write two cautious sentences for a human engineer to review.
"""

print(summary_prompt)
```

### Code Example 25. Small end-to-end Mechanical Engineering mini-project table

Purpose: create an auditable table from raw notes, local predictions, and rule-based extraction. This table can feed a report prompt.

```
project_notes = [
    "Mechanical component housing joint shows moderate cracking and minor leakage.",
    "Cooling channel inlet has severe blockage after process debris.",
    "Pressure vessel appears stable with no visible distress."
]

project_rows = []
for note in project_notes:
    pred = predict_note_priority(note)
    doc = nlp(note.lower())
    project_rows.append({
```

```
"note": note,
"clean_text": pred["clean_text"],
"priority_label": pred["label"],
"priority_probability": round(pred["probability"], 3),
"extracted_terms": " ".join([ent.text for ent in doc.ents])
})

mini_project_table = pd.DataFrame(project_rows)
mini_project_table
```

Table 30. How generated outputs become useful engineering summaries. Source: Synthesized from responsible AI workflow practice and official tool documentation.

| Input to summary    | Why it helps                             | How to prevent misuse                    |
|---------------------|------------------------------------------|------------------------------------------|
| Original notes.     | Preserve field evidence.                 | Keep notes visible next to the summary.  |
| Extracted terms.    | Make defect and severity words explicit. | Review rules and terms manually.         |
| Local model scores. | Add consistent screening information.    | Avoid treating scores as validated risk. |
| Prompt constraints. | Tell the model not to invent facts.      | Check every generated claim.             |
| Human review.       | Applies engineering judgment.            | Document revisions and approvals.        |

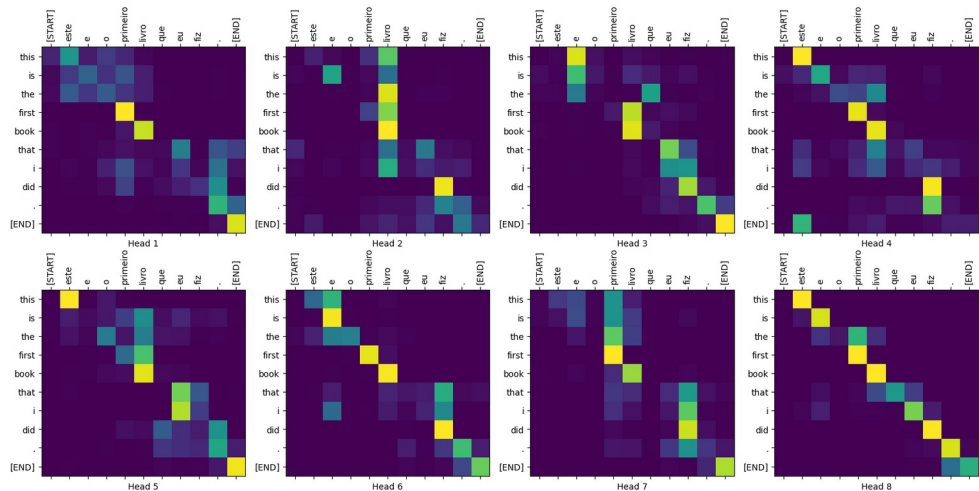


Figure 9. A complete TensorFlow attention map. Attention visualizations can help students think about which tokens or positions influence a sequence model output, although attention maps are not complete explanations. Source: TensorFlow Transformer tutorial (2024), TensorFlow documentation. Citation: TensorFlow, 2024d.

**Section summary:** A practical beginner workflow combines simple steps: clean the input, extract terms, make a local prediction, build a careful prompt, generate a draft, and review the result. The generated text should explain verified information, not replace engineering judgment.

### 13. Theory and Conceptual Background for Multimodal and API Workflows

This section collects the most important conceptual terms used in multimodal and API workflows. The purpose is to give students a clear vocabulary for reading documentation and debugging notebook examples.

Table 31. Conceptual background for API and multimodal workflows. Source: Synthesized from OpenAI API, Google Gemini API, Hugging Face, and TensorFlow documentation.

| Concept               | Beginner definition                                  | Example                                                 | Why it matters                                    |
|-----------------------|------------------------------------------------------|---------------------------------------------------------|---------------------------------------------------|
| Prompt                | Instructions and context sent to a generative model. | "Summarize these notes without inventing measurements." | A vague prompt often produces vague output.       |
| Model response        | The returned output from a model call.               | A paragraph, list, or JSON-like object.                 | Students must inspect response fields.            |
| Hosted inference call | A request sent to a remote AI service.               | API request to OpenAI, Gemini, or Hugging Face.         | Requires credentials, access, and cost awareness. |
| Multimodal input      | Input that includes more than one data type.         | Image plus text prompt.                                 | Useful for connecting photos and notes.           |

| Concept                   | Beginner definition                                     | Example                                  | Why it matters                              |
|---------------------------|---------------------------------------------------------|------------------------------------------|---------------------------------------------|
| Local inference           | Running a model in the notebook environment.            | Small TensorFlow classifier.             | Good for transparent classroom experiments. |
| API inference             | Running a model through a provider service.             | Text generation API.                     | Good for powerful pretrained systems.       |
| Proof-of-concept workflow | Small demonstration of feasibility and learning.        | Toy note classifier plus summary prompt. | Not a validated engineering product.        |
| Graceful error handling   | Code that responds helpfully when something is missing. | Print a message when API key is not set. | Prevents classroom notebooks from crashing. |

A useful mental model is:  $\text{output} = \text{model}(\text{input}, \text{settings})$ . The input may be a text prompt, image, audio file, video file, table, or combined message. The settings may include a model name, maximum tokens, or other task-specific parameters. The output may be a string, image, audio object, list of labels, or structured response. This simple input-output view is enough for beginner workflow design.

**Table 32. Prompt quality checklist for engineering summaries.** Source: Synthesized from OpenAI prompt engineering documentation, Google Gemini API prompt guidance, and engineering review practice.

| Checklist item            | Purpose                                | Example phrasing                                                             |
|---------------------------|----------------------------------------|------------------------------------------------------------------------------|
| State the role narrowly.  | Avoid broad unsupported behavior.      | "You are helping draft a review summary for a mechanical engineering class." |
| Include the raw data.     | Ground the response in visible inputs. | "Original notes: ..."                                                        |
| Request cautious wording. | Reduce overclaiming.                   | "Use cautious language and do not diagnose hidden causes."                   |
| Forbid invented details.  | Protect factual integrity.             | "Do not invent dates, measurements, asset IDs, or repair actions."           |
| Specify output format.    | Make results easy to inspect.          | "Return three bullets and one uncertainty statement."                        |
| Require review framing.   | Keep responsibility with humans.       | "Write for a human engineer to review."                                      |

Multimodal inputs need extra caution. An image may not show enough detail. A video may omit the most important moment. An audio transcript may contain recognition errors. A table may contain missing values. A generated answer that combines these inputs can sound confident even when the evidence is incomplete. Beginner workflows should explicitly include uncertainty statements and preserve the original inputs beside the generated output.

**Section summary:** The core theory is practical: prompts are instructions, responses are output objects, hosted inference calls require access, multimodal inputs combine data types, proof-of-concept workflows are not validated products, and graceful error handling keeps examples usable.

## 14. Responsible Use, Project Checklist, and Practice Activities

Mechanical Engineering applications are safety-relevant. A student demonstration can be valuable for learning, but it must not be confused with a validated inspection, design, or maintenance decision system. Generated text and model predictions should support review, not replace qualified engineering judgment.

**Table 33. Responsible use principles for beginner Mechanical Engineering AI workflows.** Source: Synthesized from responsible engineering practice and official API documentation for access, billing, and task interfaces.

| Principle                                | Beginner meaning                                                       | Practical action                                                      |
|------------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Keep original evidence visible.          | Do not hide raw notes, images, audio, or tables behind generated text. | Store inputs next to outputs in a table.                              |
| Avoid unsupported claims.                | A model should not invent causes, quantities, or recommendations.      | Prompt the model not to invent details and verify the response.       |
| Respect privacy and data rules.          | Field data may contain sensitive locations or people.                  | Use permitted classroom data and remove identifying details.          |
| Check access and pricing.                | Hosted services may cost money or have limits.                         | Read official provider pages before running paid calls.               |
| Use small baselines.                     | Simple rules and small models help reveal errors.                      | Compare API outputs with rule-based extraction and local predictions. |
| Document uncertainty.                    | AI outputs can be wrong or incomplete.                                 | Add a review note and uncertainty statement.                          |
| Do not use generated images as evidence. | Generated images are synthetic.                                        | Label synthetic media clearly in any                                  |

| Principle | Beginner meaning | Practical action |
|-----------|------------------|------------------|
|           |                  | educational use. |

**Table 34. Project checklist for an end-to-end student mini-project.** Source: Synthesized for this tutorial from the workflow structure in official spaCy, NLTK, Hugging Face, TensorFlow, Google, and OpenAI documentation.

| Step                 | Student question                                              | Deliverable                                            |
|----------------------|---------------------------------------------------------------|--------------------------------------------------------|
| Problem definition   | What small Mechanical Engineering task is being demonstrated? | One-sentence problem statement.                        |
| Input inventory      | What data types are used?                                     | List of notes, images, audio, video, or table columns. |
| Preprocessing        | How are inputs cleaned or prepared?                           | Code cell and before-after examples.                   |
| Model or tool choice | Why was this workflow selected?                               | Short explanation of local or hosted workflow.         |
| Output inspection    | How will outputs be checked?                                  | Comparison table with original inputs.                 |
| Summary generation   | What prompt creates the final draft?                          | Prompt and generated output.                           |
| Limitations          | What should not be concluded?                                 | Limitations paragraph.                                 |
| Review               | What would a human engineer need to verify?                   | Checklist of review items.                             |

**Table 35. Suggested practice activities.** Source: Synthesized for this tutorial from the concepts and workflows covered in Sections 1 through 13.

| Activity                  | What students change                                        | What students should observe                                       |
|---------------------------|-------------------------------------------------------------|--------------------------------------------------------------------|
| Rule-based extraction     | Add new asset and defect terms to the EntityRuler patterns. | More terms are extracted, but false matches may appear.            |
| Text classification       | Add more labeled inspection notes.                          | Predictions may improve when examples are more balanced.           |
| Prompt comparison         | Send the same prompt to two providers.                      | Responses differ in wording, structure, and caution.               |
| Image-to-text exploration | Caption two very different inspection images.               | Captions may be useful but incomplete.                             |
| Hosted versus local       | Compare a local summary baseline with an API draft.         | Hosted models may write better prose but still need fact checking. |
| Mini-project reflection   | Write a one-page limitations statement.                     | Students separate learning value from engineering deployment.      |

Final tutorial synthesis: spaCy and NLTK help students clean and structure engineering text. Hugging Face task interfaces help students understand pretrained workflows for text, image, audio, video, and multimodal tasks. OpenAI and Google APIs introduce hosted request-response workflows. TensorFlow supports small local models when students need transparent training and prediction. A good Mechanical Engineering proof of concept combines these tools carefully, preserves original evidence, and keeps final interpretation with human engineering review.

## 15. References

- Google AI for Developers. (2026a). Gemini API quickstart. Accessed 2026-04-24. <https://ai.google.dev/gemini-api/docs/quickstart>
- Google AI for Developers. (2026b). Using Gemini API keys. Accessed 2026-04-24. <https://ai.google.dev/gemini-api/docs/api-key>
- Google AI for Developers. (2026c). Gemini Developer API pricing. Accessed 2026-04-24. <https://ai.google.dev/gemini-api/docs/pricing>
- Google AI for Developers. (2026d). Gemini API billing. Accessed 2026-04-24. <https://ai.google.dev/gemini-api/docs/billing>
- Google AI for Developers. (2026e). Gemini API rate limits. Accessed 2026-04-24. <https://ai.google.dev/gemini-api/docs/rate-limits>
- Google AI for Developers. (2026f). Gemini API troubleshooting guide. Accessed 2026-04-24. <https://ai.google.dev/gemini-api/docs/troubleshooting>
- Hugging Face. (2026a). Transformers pipelines. Accessed 2026-04-24. [https://huggingface.co/docs/transformers/main\\_classes/pipelines](https://huggingface.co/docs/transformers/main_classes/pipelines)
- Hugging Face. (2026b). Tasks. Accessed 2026-04-24. <https://huggingface.co/tasks>
- Hugging Face. (2026c). Text generation task. Accessed 2026-04-24. <https://huggingface.co/tasks/text-generation>
- Hugging Face. (2026d). Image-to-text task. Accessed 2026-04-24. <https://huggingface.co/tasks/image-to-text>
- Hugging Face. (2026e). Text-to-speech task. Accessed 2026-04-24. <https://huggingface.co/tasks/text-to-speech>
- Hugging Face. (2026f). Automatic speech recognition task. Accessed 2026-04-24. <https://huggingface.co/tasks/automatic-speech-recognition>
- Hugging Face. (2026g). Text-to-image task. Accessed 2026-04-24. <https://huggingface.co/tasks/text-to-image>
- Hugging Face. (2026h). Image-to-image task. Accessed 2026-04-24. <https://huggingface.co/tasks/image-to-image>
- Hugging Face. (2026i). Image-text-to-text task. Accessed 2026-04-24. <https://huggingface.co/tasks/image-text-to-text>
- Hugging Face. (2026j). Any-to-any task. Accessed 2026-04-24. <https://huggingface.co/tasks/any-to-any>
- Hugging Face. (2026k). Text-to-video task. Accessed 2026-04-24. <https://huggingface.co/tasks/text-to-video>
- Hugging Face. (2026l). Video classification task. Accessed 2026-04-24. <https://huggingface.co/tasks/video-classification>
- Hugging Face. (2026m). Inference Providers. Accessed 2026-04-24. <https://huggingface.co/docs/inference-providers/en/index>
- Hugging Face. (2026n). Inference Providers pricing and billing. Accessed 2026-04-24. <https://huggingface.co/docs/inference-providers/en/pricing>
- National Vehicle systems Safety Board. (2018). Photo of cracks on Miami Mechanical component. Wikimedia Commons. Public domain. <https://commons.wikimedia.org/wiki/File:Course-generated-machined-component-crack-image>
- NLTK Project. (2025a). Natural Language Toolkit documentation. Accessed 2026-04-24. <https://www.nltk.org/>
- NLTK Project. (2025b). Sample usage for tokenize. Accessed 2026-04-24. <https://www.nltk.org/howto/tokenize.html>
- NLTK Project. (2025c). NLTK Book, Chapter 3: Processing raw text. Accessed 2026-04-24. <https://www.nltk.org/book/ch03.html>
- NLTK Project. (2025d). NLTK Book, Chapter 7: Extracting information from text. Accessed 2026-04-24. <https://www.nltk.org/book/ch07.html>
- OpenAI. (2026a). Developer quickstart. Accessed 2026-04-24. <https://developers.openai.com/api/docs/quickstart>
- OpenAI. (2026b). OpenAI API libraries. Accessed 2026-04-24. <https://developers.openai.com/api/docs/libraries>
- OpenAI. (2026c). Text generation guide. Accessed 2026-04-24. <https://developers.openai.com/api/docs/guides/text>
- OpenAI. (2026d). API pricing. Accessed 2026-04-24. <https://openai.com/api/pricing/>
- OpenAI Help Center. (2026). What is prepaid billing? Accessed 2026-04-24. <https://help.openai.com/en/articles/8264778-what-is-prepaid-billing>
- Course-generated illustrative machined-surface texture image used for this Mechanical Engineering course conversion.
- spaCy. (2025a). spaCy 101: Everything you need to know. Accessed 2026-04-24. <https://spacy.io/usage/spacy-101>
- spaCy. (2025b). Library architecture. Accessed 2026-04-24. <https://spacy.io/api>
- spaCy. (2025c). Rule-based matching. Accessed 2026-04-24. <https://spacy.io/usage/rule-based-matching>
- spaCy. (2025d). EntityRuler API documentation. Accessed 2026-04-24. <https://spacy.io/api/entityruler>
- TensorFlow. (2024a). Basic text classification. TensorFlow Core tutorials. [https://www.tensorflow.org/tutorials/keras/text\\_classification](https://www.tensorflow.org/tutorials/keras/text_classification)
- TensorFlow. (2024b). TextVectorization API documentation. [https://www.tensorflow.org/api\\_docs/python/tf/keras/layers/TextVectorization](https://www.tensorflow.org/api_docs/python/tf/keras/layers/TextVectorization)
- TensorFlow. (2024c). Classify structured data using Keras preprocessing layers. [https://www.tensorflow.org/tutorials/structured\\_data/preprocessing\\_layers](https://www.tensorflow.org/tutorials/structured_data/preprocessing_layers)
- TensorFlow. (2024d). Image classification. TensorFlow Core tutorials. <https://www.tensorflow.org/tutorials/images/classification>

- TensorFlow. (2024e). Neural machine translation with a Transformer and Keras. TensorFlow Text tutorials. <https://www.tensorflow.org/text/tutorials/transformer>
- TensorFlow. (2024f). Image captioning with visual attention. TensorFlow Text tutorials. [https://www.tensorflow.org/text/tutorials/image\\_captioning](https://www.tensorflow.org/text/tutorials/image_captioning)
- TensorFlow Text. (2024). Word embeddings. TensorFlow Text guide. [https://www.tensorflow.org/text/guide/word\\_embeddings](https://www.tensorflow.org/text/guide/word_embeddings)
- The Original Benny C. (2023). Artificial Intelligence relation to Generative Models subset, Venn diagram. Wikimedia Commons. CC BY-SA 4.0. [https://commons.wikimedia.org/wiki/File:AI\\_relation\\_to\\_Generative\\_Models\\_subset\\_venn\\_diagram.png](https://commons.wikimedia.org/wiki/File:AI_relation_to_Generative_Models_subset_venn_diagram.png)
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, N., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems. <https://arxiv.org/abs/1706.03762>
- Xu, K., Ba, J., Kiros, R., Cho, K., Courville, A., Salakhutdinov, R., Zemel, R., and Bengio, Y. (2015). Show, attend and tell: Neural image caption generation with visual attention. Proceedings of ICML. <https://arxiv.org/abs/1502.03044>